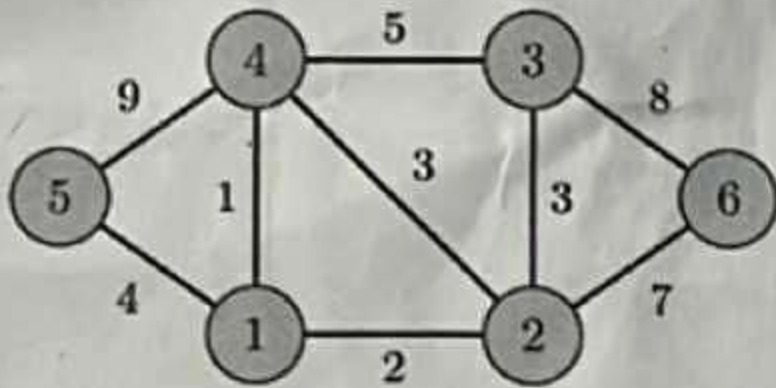


Q1. During a flash-sale event on an online grocery platform, thousands of customers place orders almost simultaneously. Each order is assigned a delivery priority score based on factors such as membership level, delivery distance, and basket freshness sensitivity. Because priority rules change dynamically during the sale, the system must repeatedly reorder all active orders in memory to always dispatch the highest-priority orders first. The platform engineers require a fast comparison-based divide-and-conquer sorting method that works in-place with minimal auxiliary memory, since sorting happens many times per minute and performance overhead must remain low. Stability of equal priority orders is not necessary. Your task is to sort the priority scores in ascending order to determine dispatch sequence. 05

Input Format: N $p_1 p_2 p_3 \dots p_N$	Output Format: <i>Sorted priority scores in ascending order on one line.</i>
Sample Input: 6 92 15 67 15 120 48	Sample Output: 15 15 48 67 92 120
Constraints: $1 \leq N \leq 10^6$ $0 \leq p_i \leq 10^{12}$ - Built-in sorting functions (like <code>sort()</code> or <code>sorted()</code>) are not allowed.	

Q2. A university campus is installing high-speed emergency fiber lines between buildings. Each connection has an associated installation cost. To minimize budget while ensuring all buildings are connected, the network team must construct a Minimum Spanning Tree (MST) — connecting every building with minimum total cable cost and no cycles. The example map below shows buildings and possible fiber routes with their respective costs. 05



Input Format: First line: number_of_nodes number_of_edges Next lines: $u \ v \ cost$	Output Format: Edges in MST: $u_1 - v_1$ $u_2 - v_2$...
Sample Input: 6 6 1 4 2 4 3 3 3 6 4 6 2 2 2 1 5 5 1 1	Sample Output: Edges in MST: 5 - 1 1 - 4 6 - 2 4 - 3 3 - 6

Q3. A smart vending machine accepts only digital payments but returns physical coins as change if the paid amount exceeds the product price. The machine has a fixed set of coin denominations available in its storage compartment. When a customer makes a purchase 05

and overpays, the system must return the minimum number of coins that sum exactly to the change amount due. Each coin can be used multiple times as long as the total value matches the required change. If it is impossible to return the exact change with the available denominations, the machine must indicate failure to dispense change. Your task is to determine the minimum number of coins required and also print one optimal sequence of coins used.

Input Format:

N M

c_1 c_2 c_3 ... c_N

N = number of coin denominations

M = total change amount to return

c_i = value of each coin denomination

Output Format:

If possible, print two lines:

-> Minimum number of coins needed

-> A space-separated sequence of coin values that sums to M using that minimum number (any one optimal sequence; order does not matter)

If impossible, print just:

-1

Sample Input:

3 11

1 5 7

Sample Output:

3

5 5 1

Constraints:

- $1 \leq N \leq 100$
- $1 \leq M \leq 10^4$
- $1 \leq c_i \leq 10^4$
- Coins may be used unlimited times